

Data Interpretation Assessment: OpenCV Pipeline Optimization

Scenario Background

An undergraduate engineering team is building an OpenCV-based real-time sorting system for an assembly line. The pipeline uses cv2.dnn.readNetFromONNX() to run an object recognition model. To optimize the system, they tested variations of the frame preprocessing scale factor, confidence thresholds, and hardware backends (cv2.dnn.DNN_BACKEND_OPENCV vs cv2.dnn.DNN_BACKEND_CUDA).

The logs from four different test runs on a sample video batch (5,000 frames total) are recorded below.

Table 1: OpenCV Pipeline Performance Logs

Backend (cv2.dnn)	Preprocessing	Confidence Threshold	Total Detection True	Positive False	Positive Total	Processing Time
Run 1 DNN_BACKEND_OPENCV	416 × 416					280.0 s 0.50 4,950 4,500
D_OPENCV						
NCV						450 42.0 s 0.75 4,100 3,950
Run 2 DNN_BACKEND_OPENCV	416 × 416					
D_OPENCV						
NCV						
Run 3 DNN_BACKEND_CUDA	416 × 416					150 41.5 s
D_CUDA						
A			0.50 4,200 3,800 400 125.0			
Run 4 DNN_BACKEND_CUDA	224 × 224					s 0.50 4,950 4,500 450
D_CUDA						
A						

Questions for Students

1. Mathematical Interpretation & Code Logic

- Calculate the Frames Per Second (FPS) achieved in Run 2 versus Run 3. • Based on these calculations, explain the exact computational advantage of switching the OpenCV backend line from:

*net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV) to
cv2.dnn.DNN_BACKEND_CUDA.*

Calculate the Frames Per Second (FPS) achieved in Run 2 versus Run

- 3. The total number of frames processed in each run is 5,000 frames.

The formula for Frames Per Second (FPS) is:

$$\text{FPS} = \text{Total Number of Frames} / \text{Total Processing Time}$$

Run 2 – OpenCV CPU Backend

Given:

- Total Frames = 5,000
- Processing Time = 280 seconds

$$\text{FPS} = 5,000 / 280$$

$$\text{FPS} \approx 17.86 \text{ FPS}$$

Therefore, Run 2 achieved approximately 17.86 FPS.

Run 3 – CUDA GPU Backend

Given:

- Total Frames = 5,000
- Processing Time = 42 seconds

$$\text{FPS} = 5,000 / 42$$

$$\text{FPS} \approx 119.05 \text{ FPS}$$

Therefore, Run 3 achieved approximately 119.05 FPS.

Comparison

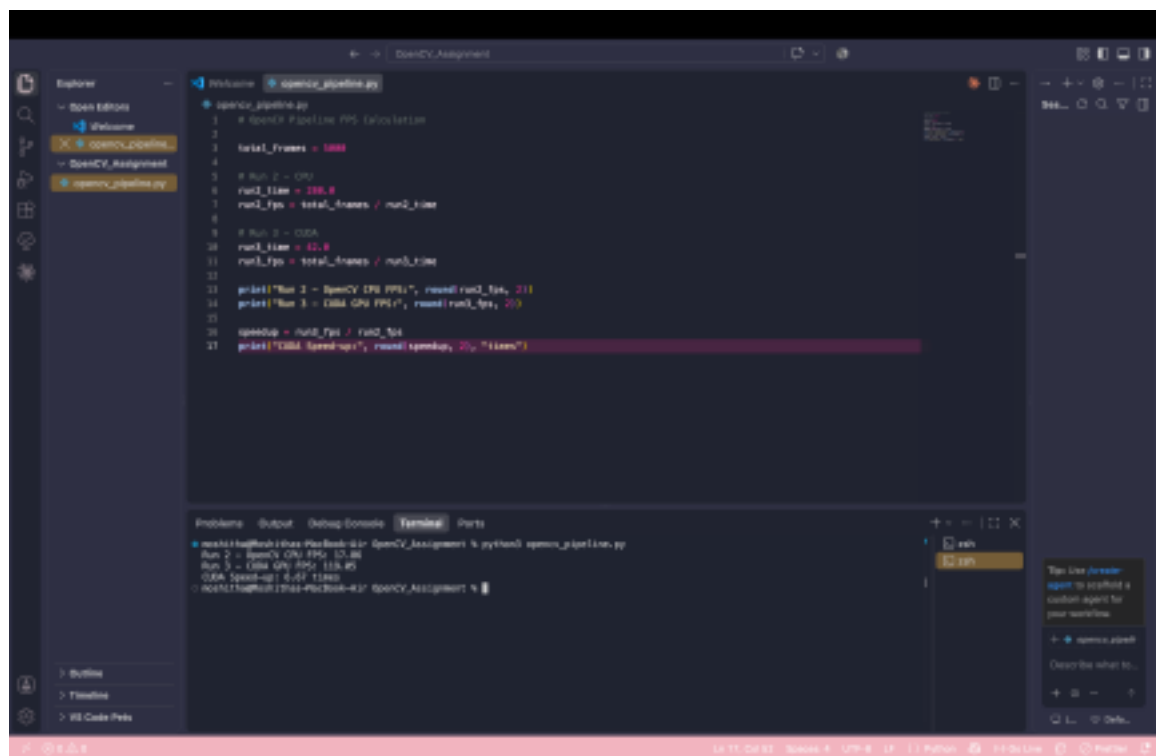
The CUDA backend increased the processing speed from approximately 17.86 FPS to 119.05 FPS.

Speed-up:

$$119.05 / 17.86 \approx 6.67\times$$

Thus, Run 3 is approximately 6.6 times faster than Run 2.

Switching from DNN_BACKEND_OPENCV to DNN_BACKEND_CUDA allows the neural-network computations, particularly highly parallel matrix multiplications and tensor convolution operations, to be executed using the GPU. This provides a major computational speed advantage. In this experiment, the processing speed increased from approximately 17.86 FPS to 119.05 FPS, giving a speed-up of approximately 6.6 $\times$, without changing the model weights or the detection accuracy settings.



```
opencv_pipeline.py
1 # OpenCV Pipeline FPS Calculation
2
3 total_frames = 1000
4
5 # Run 2 - CPU
6 run2_time = 230.8
7 run2_fps = total_frames / run2_time
8
9 # Run 3 - CUDA
10 run3_time = 42.8
11 run3_fps = total_frames / run3_time
12
13 print("Run 2 - OpenCV FPS FPS:", round(run2_fps, 2))
14 print("Run 3 - CUDA GPU FPS:", round(run3_fps, 2))
15
16 speedup = run3_fps / run2_fps
17 print("CUDA Speed-up:", round(speedup, 2), "times")

Problems Output Debug Console Terminal Ports
+ makeItHappens@MacBook-Air: ~/OpenCV_Assignments % python3 opencv_pipeline.py
Run 2 - OpenCV GPU FPS: 17.86
Run 3 - CUDA GPU FPS: 119.05
CUDA Speed-up: 6.67 times
makeItHappens@MacBook-Air: ~/OpenCV_Assignments %
```

